抽油机

实现抽油机井示功图的自动识别，助力抽油机井故障自动诊断

---

1. 概述

1.1 行业背景与痛点

目前，我国大多数的油田都是利用有杆泵抽油井采油，但由于井下地层因素复杂，抽油杆柱运动到井下抽油泵的过程中存在许多不明因素，均易引发抽油机井各种故障，一旦出现故障，不仅会造成石油开采不能有序进行、采油成本增加、降低生产抽油效率、设备损坏，甚至造成油井安全事故，引发严重的人员和财产损失，因此实时准确的对抽油机井进行诊断很有必要。

载荷(P)和位移(S)是抽油机驴头在上下往复运动时产生的参数，二者构成的闭合曲线即为示功图，它可实时反映气、油、水、砂、蜡等井内因素对抽油机工况的影响，在有杆抽油机故障过程中，可以利用获得的示功图定性地分析抽油机井地工作状态，即使调整工作参数，发现和排除故障，是一种有效的故障诊断方法。

示功图采集频繁，数据量大，传统示功图人工分析法需要巡井工人凭借自身的知识经验进行示功图识别，需要消耗大量的人力、物力，且存在受个人专业知识经验影响大、识别准确率低，识别效率低等问题，难于推广。

随着计算机技术的发展，人工神经网络得到广泛的应用，人工神经网络具有良好的非线性逼近能力，BP神经网络、RBF神经网络、小波神经网络、极限学习机、自组织神经网络等模型相继被应用到抽油机故障诊断中，正逐步取代传统的人工分析。但受限于模型的内部机制，这些方法存在以下不足：1)特征提取时直接将示功图的上百个位移−载荷数据对作为输入，造成神经网络模型内部映射结构复杂，严重影响模型的诊断精度；2）故障诊断是以示功图轮廓特征为基础，而位移−载荷的直接点对点的输入神经网络模型，无法有效提取示功图的轮廓特征。

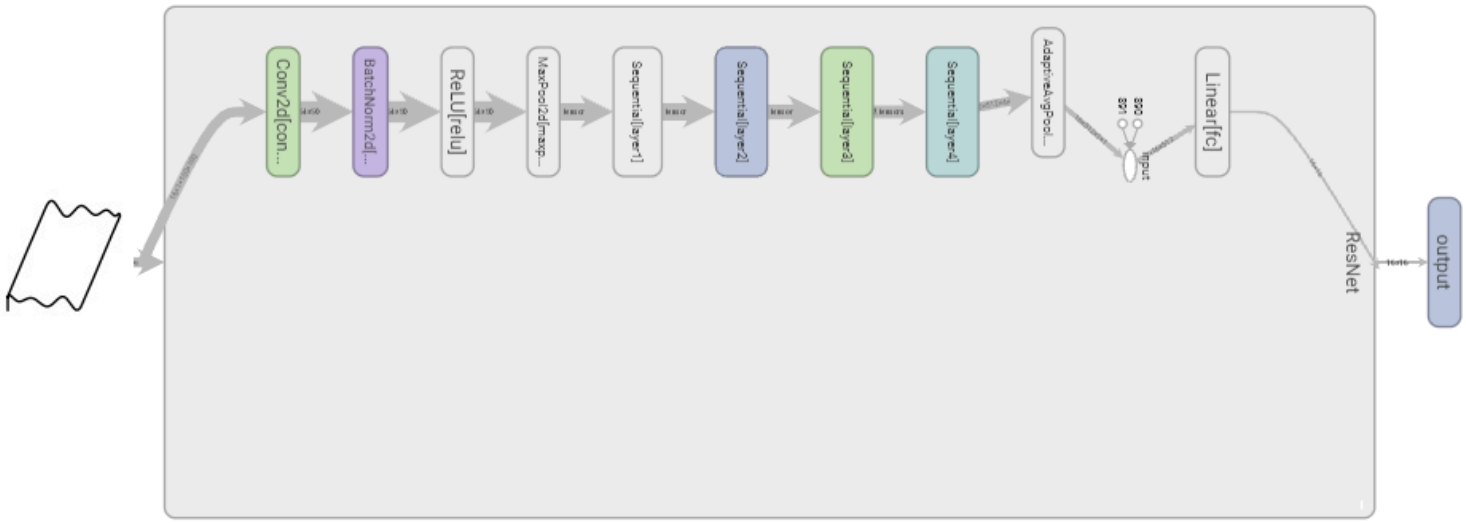
针对神经网络算法存在的问题，研究人员相继提出了多种示功图特征提取方法，如专家系统、傅里叶描述子、灰度矩阵、不变矩特征、Freeman链码、差分曲线、图形几何特征等，在提取到示功图特征后，使用支持向量机、BP神经网络、决策树、集成算法、KNN等机器学习算法进行示功图的分类识别。但上述示功图识别方法不仅需要通过各种特征工程来人工预先选取示功图特征，进而训练各种分类器完成示功图的识别，

无法实现端到端的自动识别，而且存在示功图特征选取不准确、泛化能力差、参数过多的问题，进行无法很好的训练出高效的分类器，造成识别准确率没有明显的提高。

1.2 项目价值

近年来，随着大规模数据集的不断涌现和计算机计算能力的不断提高，以卷积神经网络为代表的深度学习模型，如VGG、ResNet、DenseNet等，已成为模式识别、目标检测、自然语言处理等研究领域的研究热点。抽油机井示功图识别作为一类图像识别的在石油开采领域的具体应用，通过深度学习进行解决具有很好的适应性和可行性，通过大量的示功图数据进行深度学习，能够得到准确率较高的模型，高效的解决传统抽油机井故障诊断以及示功图识别存在的问题。

为了解决上述问题，我们采用专业的深度学习框架——PaddlePaddle，借鉴图像识别的思想，基于ResNet18模型进行训练，达到了精确识别一张示功图仅用1s的效果。在PaddlePaddle的助力下，我们通过本项目实现了抽油机井故障诊断技术与人工智能的交叉融合，反向带动了智慧油田的进步，解决了目前示功图图像识别中只能依靠有丰富石油开采知识和机械知识的专家进行识别的问题，填补了抽油机井故障诊断领域的技术空白，减少人力投入，提高石油工程施工的决策效率和质量。同时，在后续的工作中，也可以在本项目的基础上，采用迁移学习的方法，研究专门适用于地质数据分布特征的迁移学习方案，将测量勘探—迁移学习—机器学习连接为一个应用整体，能为未来把该迁移学习技术引入工业生产中，带来经济效益做铺垫。



2.实验过程

2.1 数据准备

原始的示功图悬点位移、悬点载荷坐标点数据存在于xlsx文件中，需要读取xlsx文件，使用matplotlib绘制出示功图图像

In [1]

```
import pandas as pdimport numpy as npfrom pandas import Series, DataFrameimport os import matplotlib.pyplot as pltfrom PIL import Imageimport timeimport syspath = './shigongtu_xlsx/'# 获取目录下文件files = os.listdir(path)# 获取文件名filenames = []for filename in files:    filenames.append(filename[13:-1])# 获取工况类型gongkuangTypes = []for filename in filenames:    gongkuangTypes.append(filename[:-4])# 定义保存图片函数defsave_img(gktype, i, x, y):if (os.path.exists("shigongtu/") isFalse):os.mkdir("shigongtu/")    if (os.path.exists("shigongtu/" + gktype) isFalse):os.mkdir("shigongtu/" + gktype)    plt.figure(figsize=(1, 1))    plt.axis('off')plt.rcParams['savefig.dpi'] = 100    plt.plot(y, x, color="black", linewidth=1)    path = "shigongtu/" + gktype + "/" + str(i) + ".jpg"    plt.savefig(path)    plt.close()defcreate_img():# 遍历每种工况，生成示功图for gongkuang inrange(len(gongkuangTypes)):# 打开Excel表格        excel = pd.read_excel(path + files[gongkuang])        rows, cols = excel.shape        # 将列名转化为list        cols = list(excel.columns)        # 去除第一列cols = cols[1:]        # 去除第一列后的表格        newexcel = excel.iloc[:, 1:]        # 将缺失值填充0        newexcel = newexcel.fillna(0)        # 获取所有图形的X坐标和Y坐标xs = newexcel.iloc[:, [i % 2 == 0for i inrange(len(newexcel.columns))]]        ys = newexcel.iloc[:, [i % 2 == 1for i inrange(len(newexcel.columns))]]        # 遍历生成图形for i inrange(ys.shape[1]):            x = xs.iloc[:, i]            y = ys.iloc[:, i]save_img(gongkuangTypes[gongkuang], i, y, x)            # 生成示功图# 示功图已生成，可跳过#create_img()
```

2.2 模型组网

2.2.1 定义数据预处理工具-torchvision.datasets.ImageFolder

将示功图数据集划分训练集和测试集，各工况类型训练集和测试集分别占总数据集的80%和20%，保存到train、test文件夹下

In [2]

```
import paddle.vision.transforms as T
from paddle.vision.datasets import DatasetFolder
from matplotlib import pyplot as plt
import paddle
from paddle import nn
import time
import numpy as np
import pandas as pd
import sys
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report

#定义数据预处理方法
transform  = T.Compose([
         T.Grayscale(1),
         T.Resize(100),
         T.ToTensor()
])

train_dataset = DatasetFolder('shigongtu/train', transform=transform)

test_dataset = DatasetFolder('shigongtu/test', transform=transform)

print("训练集数量：", len(train_dataset.targets))
print("测试集数量：", len(test_dataset.targets))
print("类索引：", train_dataset.class_to_idx)
idx_to_class={k:v for v,k in  train_dataset.class_to_idx.items()}

from PIL import Image
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False# print(idx_to_class)for idx
inrange(len(train_dataset.samples)):
    if idx % (len(train_dataset.targets) / 16) == 0:
        # print(train_dataset.imgs[idx][0])# print(type(train_dataset.imgs[idx][1]))
        plt.figure(figsize=(10, 12))
        infer_path= './' + train_dataset.samples[idx][0]
        img_analysis = Image.open(infer_path)
        plt.subplot(1,3,1)
        plt.imshow(img_analysis)
        plt.title(idx_to_class[train_dataset.samples[idx][1]])
```

训练集数量： 832

测试集数量： 144

类索引： {'上碰泵': 0, '下碰泵': 1, '供液不足': 2, '卡泵': 3, '固定阀损坏': 4, '固定阀漏失': 5, '抽油杆断': 6, '柱塞脱出泵筒': 7, '正常工况': 8, '气体影响': 9, '气锁': 10, '油井出砂': 11, '油管漏失': 12, '游动阀损坏': 13, '游动阀漏失': 14, '稠油影响': 15}

/opt/conda/envs/python35-paddle120-env/lib/python3.7/site-packages/matplotlib/cbook/__init__.py:2349: DeprecationWarning: Using or importing the ABCs from 'collections' instead of from 'collections.abc' is deprecated, and in 3.8 it will stop working

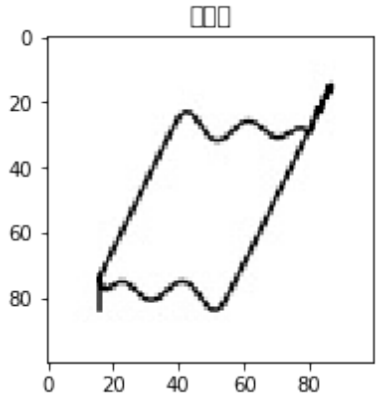
if isinstance(obj, collections.Iterator):

/opt/conda/envs/python35-paddle120-env/lib/python3.7/site-packages/matplotlib/cbook/__init__.py:2366: DeprecationWarning: Using or importing the ABCs from 'collections' instead of from 'collections.abc' is deprecated, and in 3.8 it will stop working

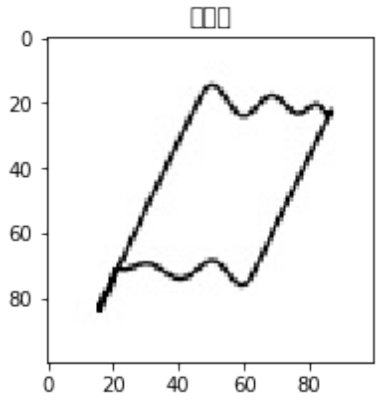
return list(data) if isinstance(data, collections.MappingView) else data

/opt/conda/envs/python35-paddle120-env/lib/python3.7/site-packages/matplotlib/font_manager.py:1331: UserWarning: findfont: Font family ['sans-serif'] not found. Falling back to DejaVu Sans

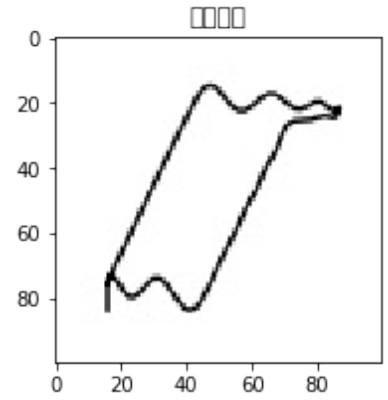
(prop.get_family(), self.defaultFamily[fontext]))



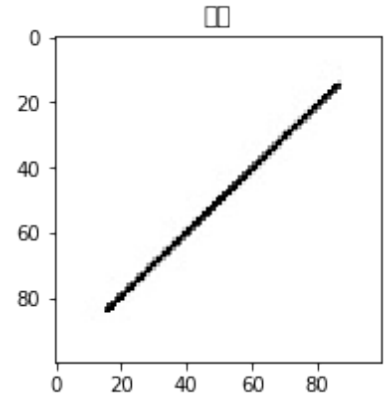
<Figure size 720x864 with 1 Axes>



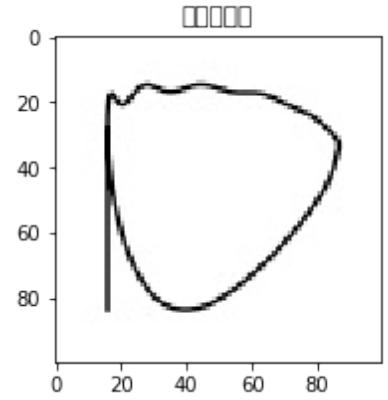
<Figure size 720x864 with 1 Axes>



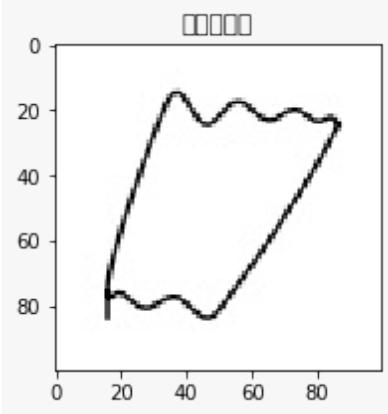
<Figure size 720x864 with 1 Axes>



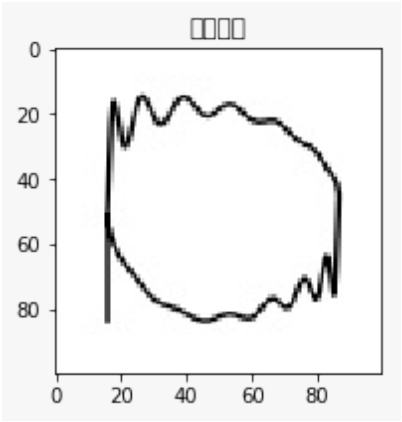
<Figure size 720x864 with 1 Axes>



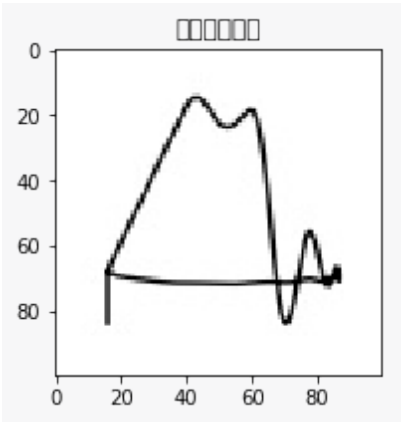
<Figure size 720x864 with 1 Axes>



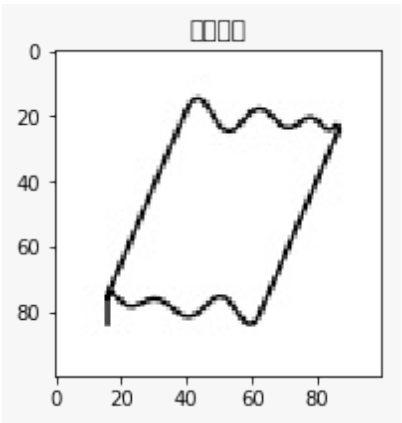
<Figure size 720x864 with 1 Axes>



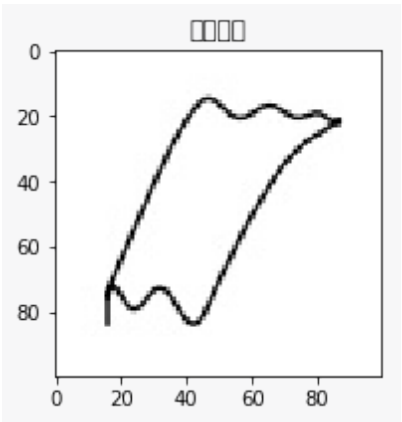
<Figure size 720x864 with 1 Axes>



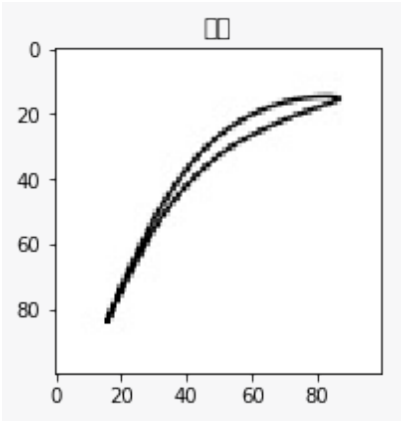
<Figure size 720x864 with 1 Axes>



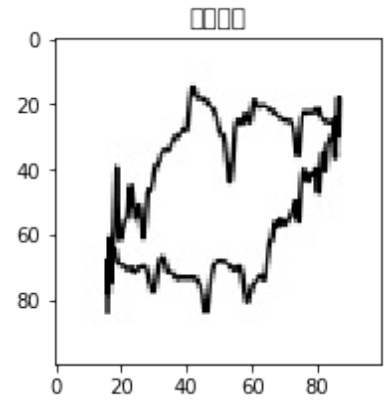
<Figure size 720x864 with 1 Axes>



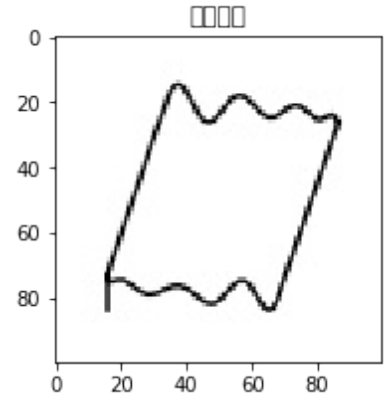
<Figure size 720x864 with 1 Axes>



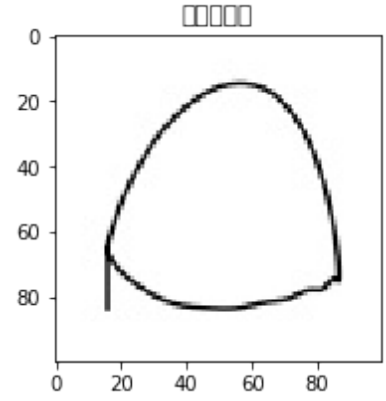
<Figure size 720x864 with 1 Axes>



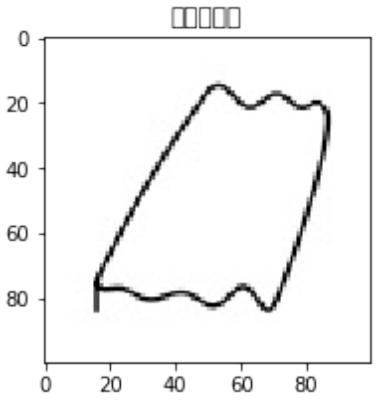
<Figure size 720x864 with 1 Axes>



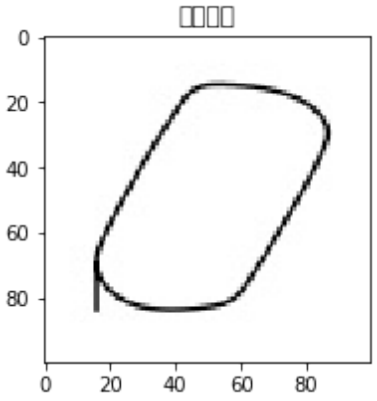
<Figure size 720x864 with 1 Axes>



<Figure size 720x864 with 1 Axes>



<Figure size 720x864 with 1 Axes>



<Figure size 720x864 with 1 Axes>

2.2.2 定义数据集加载工具-torch.utils.data.DataLoader

In [3]

```
Plain Text
```

```
from paddle.io import DataLoader# 定义batch大小batch_size = 16#定义数据集加载器train_iter = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)test_iter = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)
```

2.2.3 加载ResNet18模型并修改conv1层和全连接层

In [4]

```
Plain Text
```

```
resnet=paddle.vision.models.resnet18()# 修改模型结构
print(resnet.conv1)print(resnet.fc)resnet.conv1 = nn.Conv2D(1, 64, kernel_size=7,
stride=2, padding=3, bias_attr=False)resnet.fc = nn.Linear(in_features=512,
out_features=16, bias_attr=True)print(resnet.conv1)print(resnet.fc)
Conv2D(3, 64, kernel_size=[7, 7], stride=[2, 2], padding=3, data_format=NCHW)
Linear(in_features=512, out_features=1000, dtype=float32)
Conv2D(1, 64, kernel_size=[7, 7], stride=[2, 2], padding=3, data_format=NCHW)
Linear(in_features=512, out_features=16, dtype=float32)

```

2.2.4 定义优化器及损失函数

In [5]

```
</>                                                                    Plain Text
# 超参数learning_rate = 1e-3loss_fn = nn.CrossEntropyLoss()optimizer
=paddle.optimizer.Adam(learning_rate=learning_rate,parameters=resnet.parameters())
```

2.3 模型训练

2.3.1 定义损失值和准确率变化趋势绘图函数

In [6]

```
</>                                                                    Plain Text
iters=[]train_losses=[]train_accs=[]defdraw_train_process(iters, train_costs, train_accs):
title="training costs/training accs"    plt.title(title, fontsize=24)
plt.ylabel("cost/acc", fontsize=14)    plt.plot(iters, train_costs, color='red',
label='training costs')    plt.plot(iters, train_accs, color='green', label='training
accs')    plt.legend()    plt.grid()    plt.show()
```

2.3.2 定义训练过程

In [32]

```
global epochs_batch_countepochs_batch_count = 0deftrain_loop(dataloader, model, loss_fn, optimizer, device):    paddle.device.set_device(device)    model.train()    size = len(dataloader)    train_loss_sum,train_acc_count, n, batch_count, start = 0.0,0, 0, 0, time.time()        for batch_id, (x_data,y_data) inenumerate(dataloader):        batch_count += 1                start1=time.time()        predicts = model(x_data)        y_data64=paddle.cast(y_data, dtype='int64')        y_data64=paddle.reshape(y_data64, [y_data64.shape[0],1])        loss = loss_fn(predicts, y_data64)                acc = paddle.metric.accuracy(predicts, y_data64)                loss.backward()        optimizer.step()        optimizer.clear_grad()        train_loss_sum+=loss        train_acc_count+=int(acc*batch_size)                global epochs_batch_count        epochs_batch_count += batch_size        iters.append(epochs_batch_count)        train_losses.append(float(train_loss_sum/(batch_count)))        train_accs.append(train_acc_count/batch_count)                if batch_id % 10 == 0:            batch_loss = train_loss_sum / (batch_count)            batch_acc = acc            print('Batch [{}] Loss: {:.4f} Acc: {:.4f}'.format(batch_id, float(batch_loss[0]), float(batch_acc[0])))            n += y_data.shape[0]            print('train loss %.4f, train accuracy %.4f, time %.4f sec'              % (train_loss_sum / batch_count, acc, time.time() - start))        deftest_loop(dataloader, model, loss_fn, device):    paddle.device.set_device(device)    model.eval()    start = time.time()    batch_count=0    loss_l=[]    acc_l=[]    for batch_id, (x_data,y_data) inenumerate(dataloader):        predicts = model(x_data)    # 预测结果        y_data64=paddle.cast(y_data, dtype='int64')        y_data64=paddle.reshape(y_data64,[y_data64.shape[0],1])        # 计算损失与精度        loss = loss_fn(predicts, y_data64)        acc = paddle.metric.accuracy(predicts, y_data64)        loss_l.append(loss)        acc_l.append(acc)        batch_count+=1print(f"test loss: {float(sum(loss_l))/batch_count:>8f}, test accuracy: {float(sum(acc_l))/batch_count:>0.4f}, time: {time.time() - start:>0.4f} sec")
```

2.3.3 选择gpu并开始训练

In [33]

```
try:    cuda_is_available = len(paddle.fluid.cuda_pinned_places())>0     device ="gpu"if cuda_is_available else"cpu"except:    device="cpu"print("Using {} device".format(device))
Using cpu device

```

In [34]

</> Plain Text

```
# 定义训练epochsepochs = 5# train modelfor epoch inrange(epochs):    print(f"Epoch {epoch+1}\n-------------------------------")    start = time.time()    train_loop(train_iter(), resnet, loss_fn, optimizer, device)    test_loop(test_iter(), resnet, loss_fn, device)    end = time.time() - start    print(f"训练时间: {end}s \n")print("训练完成!")# save best model# print("save model......")saved_path='./models/resnet18.pdparams'paddle.save(resnet.state_dict(),path=saved_path)
Epoch 1
-------------------------------

/opt/conda/envs/python35-paddle120-env/lib/python3.7/site-packages/paddle/nn/layer/norm.py:641: UserWarning: When training, we now always track global mean and variance.
  "When training, we now always track global mean and variance.")

Batch [0] Loss: 0.0003 Acc: 1.0000
Batch [10] Loss: 0.0171 Acc: 1.0000
Batch [20] Loss: 0.0135 Acc: 1.0000
Batch [30] Loss: 0.0200 Acc: 1.0000
Batch [40] Loss: 0.0163 Acc: 1.0000
Batch [50] Loss: 0.0133 Acc: 1.0000
train loss 0.0130, train accuracy 1.0000, time 57.8503 sec
test loss: 15.348334, test accuracy: 0.0833, time: 2.6166 sec
训练时间: 60.477919816970825s

Epoch 2
-------------------------------
Batch [0] Loss: 0.0001 Acc: 1.0000
Batch [10] Loss: 0.0081 Acc: 1.0000
Batch [20] Loss: 0.0044 Acc: 1.0000
Batch [30] Loss: 0.0215 Acc: 1.0000
Batch [40] Loss: 0.0200 Acc: 1.0000
Batch [50] Loss: 0.0185 Acc: 1.0000
train loss 0.0182, train accuracy 1.0000, time 49.7483 sec
```

```
test loss: 14.733910, test accuracy: 0.0833, time: 2.3284 sec
训练时间: 52.089537620544434s

Epoch 3
------------------------------
Batch [0] Loss: 0.0015 Acc: 1.0000
Batch [10] Loss: 0.0300 Acc: 1.0000
Batch [20] Loss: 0.0436 Acc: 0.9375
Batch [30] Loss: 0.0340 Acc: 1.0000
Batch [40] Loss: 0.0269 Acc: 1.0000
Batch [50] Loss: 0.0256 Acc: 1.0000
train loss 0.0251, train accuracy 1.0000, time 48.2737 sec
test loss: 12.813251, test accuracy: 0.0833, time: 2.3918 sec
训练时间: 50.67737889289856s

Epoch 4
------------------------------
Batch [0] Loss: 0.0028 Acc: 1.0000
Batch [10] Loss: 0.0120 Acc: 1.0000
Batch [20] Loss: 0.0169 Acc: 1.0000
Batch [30] Loss: 0.0488 Acc: 1.0000
Batch [40] Loss: 0.0386 Acc: 1.0000
Batch [50] Loss: 0.0376 Acc: 1.0000
train loss 0.0370, train accuracy 1.0000, time 48.5583 sec
test loss: 15.373764, test accuracy: 0.0833, time: 2.2776 sec
训练时间: 50.84754800796509s

Epoch 5
------------------------------
Batch [0] Loss: 0.0081 Acc: 1.0000
Batch [10] Loss: 0.0099 Acc: 1.0000
Batch [20] Loss: 0.0405 Acc: 0.8750
Batch [30] Loss: 0.0284 Acc: 1.0000
Batch [40] Loss: 0.0809 Acc: 1.0000
Batch [50] Loss: 0.1103 Acc: 0.9375
train loss 0.1083, train accuracy 1.0000, time 48.4306 sec
test loss: 13.920447, test accuracy: 0.0833, time: 2.2811 sec
训练时间: 50.72381567955017s

训练完成!
```

2.4 进行示功图识别

2.4.1 定义识别过程

In [35]

```
deftest_model(dataloader, model, loss_fn, device):
    paddle.device.set_device(device)
    model.eval()
    batch_count = 0
    test_loss=0
    correct=0
    start=time.time()
    for batch_id, (x_data,y_data) inenumerate(dataloader):
        batch_count+=1
        predicts = model(x_data)
        y_data64=paddle.cast(y_data, dtype='int64')
        y_data64=paddle.reshape(y_data64,[y_data64.shape[0],1])
        loss = loss_fn(predicts, y_data64)
        acc = paddle.metric.accuracy(predicts, y_data64)
        correct+=acc
        test_loss +=loss
        # 打印信息print("test batch: ", batch_count )
        print("-"*10)
        print("真实结果: ", y_data)
        print("真实分类: ", [idx_to_class[int(i)] for i in y_data.numpy()])

        # print("模型输出: ", pred)print("预测标签: ", predicts.argmax(1))
        print("预测分类: ", [idx_to_class[int(i)] for i in predicts.argmax(1)])
        print()
    test_loss /= batch_count
    correct /= batch_count
    print(f"test loss: {float(test_loss):>8f}, test accuracy: {float(correct):>0.4f},
time: {time.time() - start:>0.4f} sec")
```

2.4.2 开始识别

In [36]

```
model=paddle.vision.models.resnet18()model.conv1 = nn.Conv2D(1, 64, kernel_size=7, stride=2, padding=3, bias_attr=False)model.fc = nn.Linear(in_features=512, out_features=16, bias_attr=True)model.load_dict(paddle.load('./models/resnet18.pdparams'))test_model(test_iter(),resnet,loss_fn,device)
test batch： 1
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [8 , 5 , 0 , 3 , 9 , 10, 6 , 0 , 0 , 1 , 7 , 11, 8 , 0 , 10, 1 ])
真实分类： ['正常工况', '固定阀漏失', '上碰泵', '卡泵', '气体影响', '气锁', '抽油杆断', '上碰泵', '上碰泵', '下碰泵', '柱塞脱出泵筒', '油井出砂', '正常工况', '上碰泵', '气锁', '下碰泵']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [11, 7 , 0 , 4 , 12, 13, 8 , 0 , 0 , 2 , 9 , 15, 11, 0 , 13, 2 ])
预测分类： ['油井出砂', '柱塞脱出泵筒', '上碰泵', '固定阀损坏', '油管漏失', '游动阀损坏', '正常工况', '上碰泵', '上碰泵', '供液不足', '气体影响', '稠油影响', '油井出砂', '上碰泵', '游动阀损坏', '供液不足']

test batch： 2
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [9 , 2 , 4 , 9 , 3 , 10, 2 , 7 , 4 , 8 , 5 , 1 , 4 , 11, 6 , 1 ])
真实分类： ['气体影响', '供液不足', '固定阀损坏', '气体影响', '卡泵', '气锁', '供液不足', '柱塞脱出泵筒', '固定阀损坏', '正常工况', '固定阀漏失', '下碰泵', '固定阀损坏', '油井出砂', '抽油杆断', '下碰泵']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [12, 3 , 5 , 12, 4 , 13, 3 , 9 , 5 , 11, 7 , 2 , 5 , 15, 8 , 2 ])
预测分类： ['油管漏失', '卡泵', '固定阀漏失', '油管漏失', '固定阀损坏', '游动阀损坏', '卡泵', '气体影响', '固定阀漏失', '油井出砂', '柱塞脱出泵筒', '供液不足', '固定阀漏失', '稠油影响', '正常工况', '供液不足']

test batch： 3
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [9 , 1 , 8 , 10, 5 , 8 , 3 , 7 , 8 , 5 , 1 , 6 , 6 , 11, 7 , 1 ])
真实分类： ['气体影响', '下碰泵', '正常工况', '气锁', '固定阀漏失', '正常工况', '卡泵', '柱塞脱出泵筒', '正常工况', '固定阀漏失', '下碰泵', '抽油杆断', '抽油杆断', '油井出砂', '柱塞脱出泵筒', '下碰泵']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
```

```
       [12, 2 , 11, 13, 7 , 11, 4 , 9 , 11, 7 , 2 , 8 , 8 , 15, 9 , 2 ])
预测分类： ['油管漏失', '供液不足', '油井出砂', '游动阀损坏', '柱塞脱出泵筒', '油井出砂', '固定阀损坏', '气体影响', '油井出砂', '柱塞脱出泵筒', '供液不足', '正常工况', '正常工况', '稠油影响', '气体影响', '供液不足']

test batch： 4
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [2 , 5 , 11, 10, 11, 4 , 0 , 1 , 0 , 7 , 8 , 1 , 10, 3 , 10, 8 ])
真实分类： ['供液不足', '固定阀漏失', '油井出砂', '气锁', '油井出砂', '固定阀损坏', '上碰泵', '下碰泵', '上碰泵', '柱塞脱出泵筒', '正常工况', '下碰泵', '气锁', '卡泵', '气锁', '正常工况']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [3 , 7 , 15, 13, 15, 5 , 0 , 2 , 0 , 9 , 11, 2 , 13, 4 , 13, 11])
预测分类： ['卡泵', '柱塞脱出泵筒', '稠油影响', '游动阀损坏', '稠油影响', '固定阀漏失', '上碰泵', '供液不足', '上碰泵', '气体影响', '油井出砂', '供液不足', '游动阀损坏', '固定阀损坏', '游动阀损坏', '油井出砂']

test batch： 5
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [6, 2, 3, 10, 3, 6, 5, 7, 4, 11, 5, 9, 7, 4, 5, 9])
真实分类： ['抽油杆断', '供液不足', '卡泵', '气锁', '卡泵', '抽油杆断', '固定阀漏失', '柱塞脱出泵筒', '固定阀损坏', '油井出砂', '固定阀漏失', '气体影响', '柱塞脱出泵筒', '固定阀损坏', '固定阀漏失', '气体影响']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [8 , 3 , 4 , 13, 4 , 8 , 7 , 9 , 5 , 15, 7 , 12, 9 , 5 , 7 , 12])
预测分类： ['正常工况', '卡泵', '固定阀损坏', '游动阀损坏', '固定阀损坏', '正常工况', '柱塞脱出泵筒', '气体影响', '固定阀漏失', '稠油影响', '柱塞脱出泵筒', '油管漏失', '气体影响', '固定阀漏失', '柱塞脱出泵筒', '油管漏失']

test batch： 6
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [1 , 11, 0 , 0 , 6 , 3 , 6 , 5 , 7 , 8 , 2 , 3 , 6 , 5 , 8 , 3 ])
真实分类： ['下碰泵', '油井出砂', '上碰泵', '上碰泵', '抽油杆断', '卡泵', '抽油杆断', '固定阀漏失', '柱塞脱出泵筒', '正常工况', '供液不足', '卡泵', '抽油杆断', '固定阀漏失', '正常工况', '卡泵']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [2 , 15, 0 , 0 , 8 , 4 , 8 , 7 , 9 , 11, 3 , 4 , 8 , 7 , 11, 4 ])
预测分类： ['供液不足', '稠油影响', '上碰泵', '上碰泵', '正常工况', '固定阀损坏', '正常工况', '柱塞脱出泵筒', '气体影响', '油井出砂', '卡泵', '固定阀损坏', '正常工况', '柱塞脱出泵筒', '油井出砂', '固定阀损坏']

test batch： 7
----------
```

```
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [3 , 11, 9 , 1 , 9 , 0 , 4 , 8 , 6 , 2 , 1 , 7 , 0 , 2 , 10, 9 ])
真实分类： ['卡泵', '油井出砂', '气体影响', '下碰泵', '气体影响', '上碰泵', '固定阀损坏', '正常工况', '抽油杆断', '供液不足', '下碰泵', '柱塞脱出泵筒', '上碰泵', '供液不足', '气锁', '气体影响']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [4 , 15, 12, 2 , 12, 0 , 5 , 11, 8 , 3 , 2 , 9 , 0 , 3 , 13, 12])
预测分类： ['固定阀损坏', '稠油影响', '油管漏失', '供液不足', '油管漏失', '上碰泵', '固定阀漏失', '油井出砂', '正常工况', '卡泵', '供液不足', '气体影响', '上碰泵', '卡泵', '游动阀损坏', '油管漏失']

test batch： 8
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [9 , 11, 2 , 11, 7 , 5 , 0 , 3 , 3 , 11, 2 , 11, 6 , 6 , 4 , 2 ])
真实分类： ['气体影响', '油井出砂', '供液不足', '油井出砂', '柱塞脱出泵筒', '固定阀漏失', '上碰泵', '卡泵', '卡泵', '油井出砂', '供液不足', '油井出砂', '抽油杆断', '抽油杆断', '固定阀损坏', '供液不足']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [12, 15, 3 , 15, 9 , 7 , 0 , 4 , 4 , 15, 3 , 15, 8 , 8 , 5 , 3 ])
预测分类： ['油管漏失', '稠油影响', '卡泵', '稠油影响', '气体影响', '柱塞脱出泵筒', '上碰泵', '固定阀损坏', '固定阀损坏', '稠油影响', '卡泵', '稠油影响', '正常工况', '正常工况', '固定阀漏失', '卡泵']

test batch： 9
----------
真实结果： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=True,
       [10, 7 , 4 , 10, 7 , 4 , 4 , 5 , 2 , 8 , 4 , 2 , 10, 9 , 0 , 9 ])
真实分类： ['气锁', '柱塞脱出泵筒', '固定阀损坏', '气锁', '柱塞脱出泵筒', '固定阀损坏', '固定阀损坏', '固定阀漏失', '供液不足', '正常工况', '固定阀损坏', '供液不足', '气锁', '气体影响', '上碰泵', '气体影响']
预测标签： Tensor(shape=[16], dtype=int64, place=CPUPlace, stop_gradient=False,
       [13, 9 , 5 , 13, 9 , 5 , 5 , 7 , 3 , 11, 5 , 3 , 13, 12, 0 , 12])
预测分类： ['游动阀损坏', '气体影响', '固定阀漏失', '游动阀损坏', '气体影响', '固定阀漏失', '固定阀漏失', '柱塞脱出泵筒', '卡泵', '油井出砂', '固定阀漏失', '卡泵', '游动阀损坏', '油管漏失', '上碰泵', '油管漏失']

test loss: 13.920448, test accuracy: 0.0833, time: 2.3466 sec

```

3.实验结果与分析

In [37]

```
draw_train_process(iters, train_losses, train_accs)

# tensorboard模型可视化# from torch.utils.tensorboard import SummaryWriter# # default `log_dir` is "runs" - we'll be more specific here# writer = SummaryWriter('runs/shigongtu_experiment')# dataiter = iter(train_iter)# images, labels = dataiter.next()# print(images.shape)# writer.add_graph(resnet, images)# writer.close()
```

/opt/conda/envs/python35-paddle120-env/lib/python3.7/site-packages/matplotlib/font_manager.py:1331: UserWarning: findfont: Font family ['sans-serif'] not found. Falling back to DejaVu Sans

(prop.get_family(), self.defaultFamily[fontext]))

<Figure size 432x288 with 1 Axes>

从上面的损失曲线和准确率曲线可以看出，PaddlePaddle的ResNet18模型经过一开始少量的训练就能很快达到将近80%的准确率，经过训练后，准确率稳定在99%左右，损失也有一定的下降。

In [38]

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplayfrom sklearn.metrics import classification_reporttest_y = np.zeros((0,), dtype=np.int)test_pred = np.zeros((0,), dtype=np.int)resnet.eval()for X, y in test_iter:    pred = resnet(X)    test_y = np.append(test_y, y.numpy(), axis = 0)    test_pred = np.append(test_pred, pred.argmax(1).numpy(), axis = 0)    # 混淆矩阵confusion_matrix(test_y, test_pred)
array([[12,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0],       [ 0,  0, 12,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0],       [ 0,  0,  0, 12,  0,  0,  0,  0,  0,
```

```
0,  0,  0,  0,  0,  0],       [ 0,  0,  0,  0, 12,  0,  0,  0,  0,  0,  0,  0,  0,  0,
0],       [ 0,  0,  0,  0,  0, 12,  0,  0,  0,  0,  0,  0,  0,  0,  0],       [ 0,  0,  0,
0,  0,  0,  0, 12,  0,  0,  0,  0,  0,  0,  0],       [ 0,  0,  0,  0,  0,  0,  0,  0, 12,
0,  0,  0,  0,  0,  0],       [ 0,  0,  0,  0,  0,  0,  0,  0,  0, 12,  0,  0,  0,  0,
0],       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 12,  0,  0,  0],       [ 0,  0,  0,
0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 12,  0,  0],       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,
0,  0,  0,  0, 12,  0],       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
12],       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0],       [ 0,  0,
0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0],       [ 0,  0,  0,  0,  0,  0,  0,  0,
0,  0,  0,  0,  0,  0,  0]])
```

In [39]

</> Plain Text

```
cm = confusion_matrix(test_y, test_pred)disp =
ConfusionMatrixDisplay(confusion_matrix=cm)disp.plot()
---------------------------------------------------------------------------
TypeErrorTraceback (most recent call
last)/tmp/ipykernel_1725/4116286572.pyin<module>1cm=confusion_matrix(test_y,test_pred)2---
-> 3disp=ConfusionMatrixDisplay(confusion_matrix=cm)4disp.plot()TypeError: __init__()
missing 1 required positional argument: 'display_labels'
```

In []

</> Plain Text

```
# 分类报告report = classification_report(test_y, test_pred)print(report)
```

总结

在PaddlePaddle框架的助力下，我们克服了通过人工提取示功图特征以对抽油机井故障进行诊断的限制，解决了传统示功图图像识别准确率低、特征提取复杂、泛化性能差的问题，实现端到端的抽油机井故障自动诊断。本次实验只使用较少的数据作为训练集，且各类数据类内差距小，类间差距大，在实际大数据集的应用场景下，配合更加庞大的示功图数据集以及各种复杂的示功图工况类型，我们相信，一定可以实现更加准确的识别。在该示功图识别的基础上，未来可以实现根据连续采集的示功图来预测未来示功图工况类型的目的，做到抽油机井故障类型的提前判断，进一步提高学科融合的意义。